



SWE 3513

Artificial Intelligence

Assignment 1

Musanze Cooperative Harvest and Dispatch Decision Lab

A scenario based machine learning group assignment

Assessment detail	Requirement
Group size	4 to 5 students
Coverage	Days 1 to 4 only
Issued	Tuesday, 29 September 2026
Deadline	Wednesday, 30 September 2026 at 5:00 PM CAT (Africa/Kigali)
Total	100 marks
Submission	One group package plus individual verification

Purpose and learning outcomes

Your group will build a reproducible decision pipeline for a fictional potato cooperative. The assignment assesses the practical content from Days 1 to 4: numerical representation with NumPy, regression from first principles, supervised classification, and unsupervised clustering.

Load, validate, clean, and vectorize tabular data without hard-coded outputs.

Implement and explain linear regression with gradient descent using NumPy.

Train and evaluate a classification model with appropriate metrics.

Standardize features, select a clustering configuration, and interpret clusters cautiously.

Package the analysis so an assessor can rerun it on an unseen file with the same schema.

Scenario

Musanze HarvestLink Cooperative coordinates potato collection from several fictional farms. Before dispatch, the operations team needs three decisions: estimate expected harvest weight, flag consignments needing dispatch attention, and group collection points with similar operating profiles. Your group will deliver a command line pipeline and a simple interface concept for the staff who consume its results.

Use only the lecturer issued group dataset named AI_A1_GXX.csv. The dataset is fictional and group specific. Do not rename columns, merge another group's data, or replace the file with generated values. The pipeline must calculate and print the dataset SHA-256 fingerprint.



Dataset schema

Field	Type	Use
record_id	text	Unique row identifier; never use as a model feature
plot_area_ha	numeric	Farm area
rainfall_mm	numeric	Recent rainfall estimate
soil_ph	numeric	Soil acidity measure
seed_kg	numeric	Seed quantity
distance_km	numeric	Distance to collection point
arrival_hour	numeric	Planned arrival hour in 24-hour form
actual_yield_kg	numeric target	Regression target
dispatch_attention	binary target	Classification target: 0 or 1

The analytical product your group must build

Data and vectorization pipeline. Load the CSV, validate column names and types, report missing and duplicate values, separate identifiers from features, and produce a NumPy feature matrix. Save data_report.json with row count, feature count, missing values, descriptive statistics, group code, and SHA-256 fingerprint.

Regression from first principles. Implement linear regression and batch gradient descent using NumPy. Do not use a library estimator for this section. Include feature scaling, train/test split, loss history, predictions, MAE, RMSE, and R-squared. Save regression_metrics.json and regression_loss.png.

Classification. Use pandas and scikit-learn to train one interpretable classifier for dispatch_attention. Use a stratified split when possible. Report confusion matrix, accuracy, precision, recall, and F1. Explain which error is more costly in this scenario. Save classification_metrics.json and confusion_matrix.png.

Clustering. Using only input features, standardize the data, evaluate k from 2 through 5 with silhouette score, justify the selected k, and save a cluster label for every record. Save clustering_metrics.json, clusters.csv, and cluster_plot.png. Do not claim that a cluster is a verified real-world category.

Reusable prediction command. predict.py must accept one JSON record using the documented schema and print valid JSON containing regression



prediction, classification prediction and probability, cluster label, group code, and model version. It must reject missing or malformed fields clearly.

UI and UX design PDF

Submit AI_A1_GXX_UIUX.pdf as a five to seven page PDF for a proposed staff decision dashboard. You are not required to code this interface. It must show how a non-technical user would understand and act on the model outputs.

Page	Required content	Assessment evidence
1	Cover and operational problem	Group code, members, stakeholder and three decisions
2	User journey and information flow	From CSV upload to review, prediction, and human decision
3 to 4	Annotated wireframes	Data quality, prediction, classification warning, and clusters
5	Responsible AI states	Uncertainty, missing data, model limitation, human override
6 to 7 if used	Visual system and rationale	Accessible colors, labels, legends, and three justified decisions

Required command interface

The project must run from a clean environment with the following commands. Paths may be adapted only if README.md gives an equivalent single command. Notebook-only submissions are not accepted.

```
python run_all.py --data data/AI_A1_GXX.csv --output artifacts/ --group AI-GXX
python predict.py --record '{"plot_area_ha":1.2,"rainfall_mm":81,"soil_ph":5.7,"seed_kg":210,"distance_km":14,"arrival_hour":9}'
```

Team roles and individual responsibility

Assign one named student to each role before analysis. In a four member group, Member 4 also owns reproducibility and release evidence. In a five member group, Member 5 owns those tasks.

Role	Primary responsibilities	Required personal evidence
Member 1 Data and UX lead	Schema checks, vectorization, data report, decision-dashboard PDF coordination	Data functions; PDF pages owned; at least two meaningful commits



Role	Primary responsibilities	Required personal evidence
Member 2 Regression engineer	NumPy model, gradient descent, scaling, loss curve and regression metrics	Implementation and tests; derivation notes; at least two meaningful commits
Member 3 Classification engineer	Split, classifier, metrics, confusion matrix, cost of errors	Model and evaluation files; at least two meaningful commits
Member 4 Clustering and QA engineer	Scaling, k comparison, silhouette scores, cluster outputs, cross-pipeline QA	Clustering files and test log; at least two meaningful commits
Member 5 Reproducibility and release lead	CLI contract, predict command, requirements, README, evidence and final package	Clean-run record; final hash; at least two meaningful commits

Technical and scientific constraints

Use Python, NumPy, pandas, scikit-learn, matplotlib, and seaborn. No AutoML service or prebuilt regression estimator may replace the NumPy gradient descent section.

Use a fixed random seed and record it in every relevant metrics file. Fit scalers only on training data for supervised models.

Do not tune on the test set. Do not use `actual_yield_kg` or `dispatch_attention` as clustering inputs.

Do not hard-code metrics, predictions, cluster labels, paths, or row counts. The assessor will run the pipeline with a hidden CSV that preserves the schema but changes the rows.

The complete pipeline must finish without manual notebook steps and must overwrite generated artifacts safely in the specified output folder.



INSTITUT D'ENSEIGNEMENT SUPÉRIEUR DE RUHENGARI

B.P. 155, Ruhengeri, Rwanda

T : +250 788 90 30 30, +250 788 90 30 32, W : www.ines.ac.rw, E : info@ines.ac.rw

Marking rubric

Criterion	Marks	Full credit evidence
Data pipeline and vectorization	15	Robust schema checks, correct feature matrix, fingerprint and complete data report
NumPy regression	20	Correct gradient descent, scaling, split, loss behavior, predictions and metrics
Classification	15	Valid evaluation, all specified metrics, confusion matrix and cost interpretation
Clustering	15	Correct scaling, k comparison, justified choice, labels and cautious interpretation
UI and UX PDF	10	Clear decision workflow, accessible wireframes, uncertainty and human override
Individual role evidence	10	Traceable owned work, meaningful commits, signed contribution mapping
Live individual verification	10	Explains the method and completes a small unseen test or change
Peer accountability	5	Confidential ratings are consistent with repository and contribution evidence
Total	100	75 team marks plus 25 individual marks



Assessor verification script

The assessor will use the same sequence for every group. A hidden file with the published schema will be used to confirm that outputs are computed rather than hard-coded.

Test	Observable result	Linked marks
1 Clean setup	Dependencies install and README commands are complete	Reproducibility
2 Group data	Pipeline prints correct group code and SHA-256; artifacts match the issued CSV	Data
3 Hidden data	Pipeline runs on unseen rows without code editing or fixed row assumptions	All technical
4 Regression	Loss and metrics are recomputed; predictions are finite and plausible	Regression
5 Classification	Metrics and confusion matrix are recomputed from predictions	Classification
6 Clustering	Scores for k 2 to 5 appear and every input record receives a label	Clustering
7 Predict command	Valid JSON is returned; malformed or missing fields are rejected	Reliability
8 Leakage check	Targets and identifiers are excluded where required; scaler use is correct	Scientific quality
9 Evidence	PDF, commits, roles, metrics, video, dataset hash and final hash agree	Individual evidence
10 Live change	Selected member changes seed, threshold, learning rate, or k range and explains the result	Live verification

Submission Package

The assignment must be submitted through the **Moodle assignment activity** no later than **Wednesday, 30 September 2026 at 5:00 PM CAT**. Only the designated group leader should submit on behalf of the group.

Upload the following three files:

1. AI_A1_GXX.zip — complete Python project, dataset and generated evidence
2. AI_A1_GXX_UIUX.pdf — proposed decision-dashboard UI/UX document
3. AI_A1_GXX_CONTRIBUTIONS.pdf — signed contribution and accountability record



Replace GXX with the lecturer-assigned group number. For example, Group 04 should submit AI_A1_G04.zip.

The ZIP file must contain the following structure:

```
AI_A1_GXX/
├── README.md
├── requirements.txt
├── run_all.py
├── predict.py
├── src/                                Reusable Python modules
├── data/
│   └── AI_A1_GXX.csv                  Lecturer-issued group dataset
├── artifacts/
│   ├── data_report.json
│   ├── regression_metrics.json
│   ├── regression_loss.png
│   ├── classification_metrics.json
│   ├── confusion_matrix.png
│   ├── clustering_metrics.json
│   ├── clusters.csv
│   └── cluster_plot.png
├── models/                            Saved models and preprocessing objects
├── evidence/
│   ├── AI_A1_GXX_DEMO.mp4
│   ├── AI_USE.md
│   └── TEST_LOG.pdf
```

Do not include virtual environments, Python cache folders, notebook checkpoints, build caches, API keys, passwords, duplicated datasets, or unrelated files. Examples of prohibited folders include .venv, venv, __pycache__, and .ipynb_checkpoints.

In Moodle's **Online Text** field, provide:

Group number:

Group verification code:

Group leader:

Group members:

GitHub repository URL:

Final commit hash:

Dataset SHA-256 fingerprint:

Demonstration video link, if the video is not included in the ZIP:

Required Demonstration Video

Submit one continuous and unedited MP4 video lasting no more than 90 seconds. The terminal clock, group verification code, dataset fingerprint, and complete verification sequence must be visible.

The student operating the demonstration must introduce themselves by stating their name, registration number, and assigned group role.

The video must demonstrate the following:

1. Show the group verification code, current terminal clock, final Git commit hash, and dataset SHA-256 fingerprint.



2. Run the complete pipeline from the command line using:
`python run_all.py --data data/AI_A1_GXX.csv --output artifacts/ --group AI-GXX`
3. Show that the required JSON, CSV, PNG, and model files were generated by the command.
4. Open one metrics JSON file and one generated plot.
5. Explain one model result in the group's own words without reading from a prepared script.
6. Run `predict.py` using one valid input record and show the returned JSON result.
7. Run `predict.py` using one deliberately incomplete or malformed record and show that the program returns a clear validation error.
8. Finish by showing the filenames of the UI/UX PDF and signed contribution document.

A narrated slideshow, edited compilation, notebook screenshot, pre-recorded output, or demonstration that does not execute the submitted code will not be accepted as functional evidence.

Submission Rules

- Only the group leader submits the assignment through Moodle.
- Moodle's recorded submission time is the official submission time.
- Only the final submission received before **5:00 PM CAT** will be marked.
- Email and WhatsApp submissions will not be accepted unless a Moodle failure was formally reported to the lecturer before the deadline.
- The GitHub commit hash entered in Moodle must correspond to the exact code contained in the submitted ZIP file.
- The lecturer-issued dataset must not be renamed, replaced, manually extended, or modified.
- The dataset SHA-256 fingerprint reported in Moodle must match the fingerprint generated by `run_all.py`.
- `README.md` must provide exact environment-setup and execution commands, the tested Python version, group members, assigned roles, repository URL, final commit hash, expected outputs, and known limitations.
- `requirements.txt` must contain every external Python dependency required to run the project. The assessor will not identify or install undocumented packages.
- `AI_A1_GXX_CONTRIBUTIONS.pdf` must be signed by every group member. It must connect each requirement to the responsible student, source files, branch, commit hashes, and supporting evidence.
- `AI_USE.md` must disclose every generative AI tool used, the purpose for which it was used, important prompts or requests, affected files, and how the group tested the generated output. Write No AI used when applicable.
- All files inside `artifacts/` must be generated by the submitted code. Manually created or hard-coded metrics, predictions, charts, and cluster labels will receive no marks.
- Notebook-only submissions are not accepted. The complete workflow must run from the command line without manual notebook execution.



- The assessor may run the submitted pipeline using an unseen CSV file with the same published schema. Hard-coded row counts, file paths, metrics, predictions, and outputs will therefore fail verification.
- The assessor will not repair paths, rewrite code, rename files, add missing data, correct the dataset schema, or debug the submission.
- A project that cannot run using the instructions in README.md will receive no marks for the affected technical criteria.
- Missing, corrupted, inaccessible, or incorrectly named files will be treated as not submitted.
- Every group member must attend the live verification and be ready to explain or modify the section assigned to them.

Authenticity and responsible use of AI

This is an evidence based assessment. Generative AI may be used for explanation, brainstorming, or debugging only when its use is disclosed. It does not replace authorship, testing, or the ability to explain the submitted work.

Required AI disclosure. Submit AI_USE.md. For every tool used, record the tool name, purpose, important prompts or requests, files affected, and how the group verified the result. Write 'No AI used' if applicable.

Required process evidence. Each member must make meaningful version control commits and identify the files or decisions for which they are responsible. Renamed, duplicated, or cosmetic commits do not count.

Required verification. Every member must attend the AI live check and answer a question or perform a small change connected to their role.

Unverifiable work. A member who cannot explain or modify their claimed contribution receives zero for the individual verification criteria, even when the group product runs.

Academic misconduct. Fabricated data, falsified commits, staged peer records, copied projects, or concealed third party authorship will be referred under the institution's academic integrity rules.

Final checklist

The ZIP, folder, dataset, PDF, and video names use the correct group number. run_all.py and predict.py work exactly as documented from a clean environment.

The issued dataset is unchanged and its SHA-256 appears in data_report.json.

All required JSON, CSV, and PNG artifacts are present and were generated by the submitted code.

Every member has signed CONTRIBUTIONS.pdf and can explain or modify their own section.

The final package is submitted by Wednesday, 30 September 2026 at 5:00 PM CAT.